AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Lists and indexing

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

Before you start

What values does range(3) supply? How many iterations occur?

Plan time to read, investigate and explain. Keep predictions separate from observations.

STUDENT LESSON · 1 OF 18

Your mission

> Start with the target: Distinguish a list index from the value stored at that position. Try the recall before opening notes.

Your learning target

I can distinguish a **list index** from the value stored at that position.

I can read and update an **item** using a valid **index**.

I can trace a loop through **list** values and test an empty **list**.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Boolean conditions. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from From a paper algorithm to Python. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Distinguish a **list index** from the value stored at that position

Read and update an **item** using a valid **index**

Trace a loop through **list** values and test an empty **list**

Retrieve what you already know

> What values does range(3) supply? How many iterations occur?

My starting explanation

This week's end goal

Trace **list**s and **index**ing and explain the changed program before running it.

Notice the evidence

> Investigate the supplied Lists and indexing case. Separate observations, given facts and assumptions before choosing an explanation.

A fictional supply shelf has three trays with counts 4, 7 and 4 in that order. Store them as counts = [4, 7, 4]. Square brackets enclose the **list** and commas separate its **item**s.

The order matters: the first and third trays contain equal values but occupy different positions.

Python counts positions from zero. counts[0] reads the first **item**, counts[1] the second and counts[2] the third. len(counts) is 3 because there are three **item**s. It is a count, not the last nonnegative **index**.

This lesson uses integer counts and nonnegative indices; Python also supports negative indices, which are outside our current **index**ing convention.

> **List the given facts and one fact that is not supplied.**

STUDENT LESSON · 3 OF 18

See how it works

> Follow the mechanism for Lists and indexing. Connect each labelled input, deciding step and result.

Positions and stored counts

TRAY POSITION	INDEX	VALUE
First	0	4
Second	1	7
Third	2	4

Equal stored values do not imply equal positions. There is no item at index 3.

```
counts = [4, 7, 4]
print(len(counts))
print(counts[0])
print(counts[1])
counts[1] = 9
print(counts)
```

The printed lines are 3, 4, 7 and [4, 9, 4]. The assignment counts[1] = 9 replaces the **item** at **index** 1; it does not add a fourth **item** or change the two other values. The **length** stays 3.

A recorded count can be changed by code without proving that the physical tray really contains that many objects.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

Compare the cases

> Compare the supplied cases for Lists and indexing. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Change one thing in the pictured case

Change one thing

Change: Replace the second print with print(counts[3]).

Predicted result

Prediction: The first print shows 6, then **IndexError** stops the script before the **length** print.

Why the result follows

Reason: The attempted position does not exist.

STUDENT LESSON · 5 OF 18

Words that unlock the idea

> Use the vocabulary of Lists and indexing to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
List	An ordered collection of items
Index	An integer position used to access an item
Length	The number of items in a collection
Item	One value stored in a collection
IndexError	An error raised when an index is outside a sequence

Use two terms to explain a step in this week's example.

STUDENT LESSON · 6 OF 18

Follow a worked example

> Follow the worked trace for Lists and indexing, predict a step, then compare with its explanation.

```
counts = [4, 7, 4]
total = 0
for count in counts:
    total = total + count
print(total)
```

Item read as count	total before	total after
4	0	4
7	4	11
4	11	15

This fresh script prints 15. for count in counts supplies the stored values, not the indices. Both copies of 4 are **process**ed because both are **item**s. For an empty **list** written [], the body runs zero times and total stays 0.

For [7], it runs once and prints 7.

> **Change only the list to [2, 0, 5]. Trace the three additions and state the final output. Explain why the middle item is processed even though its value is zero.**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Lists and indexing

Indexes are positions, not values. A **list** of shelf counts is read by position.

The facts we will use

Supplied facts

Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).



Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / PYTHON-BRIDGE/7

Lists and indexing

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Draw positions

Indexes 0,1,2 contain 6,1,8 respectively.



2. Read two positions

Index 0 returns 6; index -1 returns the final value 8.



3. Check length boundary

Length is 3, but index 3 is beyond the final valid positive index 2.

CHANGE ONE THING

Replace the second print with print(counts[3]).

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Draw positions	Indexes 0,1,2 contain 6,1,8 respectively.
2. Read two positions	Index 0 returns 6; index -1 returns the final value 8.
3. Check length boundary	Length is 3, but index 3 is beyond the final valid positive index 2.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

List positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

What this example does not establish

Keep the stated limit

A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

Keep the stated limit

Read the labelled picture: Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

Keep the stated limit

Change one thing: Replace the second print with print(counts[3]). Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Lists and indexing claim and explain a repair using the supplied evidence.

```
counts = [4, 7, 4]
print(counts[3])
```

Check the claim against the facts

This deliberately faulty script raises **IndexError**. The **list** has three **item**s, but its nonnegative indices are 0, 1 and 2. Replace 3 with 2 to read the third **item**, which is 4.

For any nonempty **list**, len(counts) - 1 identifies the last **item** using a nonnegative **index**. An empty **list** has no valid **item index**.

Avoid confusing values with positions: for count in counts followed by counts[count] would try indices 4, 7 and 4 in this example. If you need both, use for **index** in range(len(counts)) and read counts[**index**].

If you only need a total, looping directly through values is simpler.

Locate the earliest unsupported step

A learner loops with for count in counts and tries to read counts[count] for counts = [4, 7, 4]. Identify the first attempted **index** and error. Explain how to print each value directly.

> **State one repair and the checks you would rerun.**

STUDENT LESSON · 8 OF 18

Practise together

> Practise Lists and indexing with support, then explain the deciding step in your own words.

For counts = [4, 7, 4], give the **length**, valid nonnegative indices and value at **index** 1. Explain how the two **item**s with value 4 differ.

> **Change only the list to [2, 0, 5]. Trace the three additions and state the final output. Explain why the middle item is processed even though its value is zero.**

Explain your prediction to a partner before comparing results.

Practise with less help: Trace an update

Use workbook W2 for the connected example "**List**s and **index**ing". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for **List**s and **index**ing. Proposed explanation: "**Length** is the last nonnegative **index**". Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate Lists and indexing. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

Start with counts = [4, 7, 4]. Run counts[0] = 6. Write the resulting **list**, its **length** and the value at **index** 2. Predict before running.

```
counts = [4, 7, 4]
print(len(counts))
print(counts[0])
print(counts[1])
counts[1] = 9
print(counts)
```

```
counts = [4, 7, 4]
total = 0
for count in counts:
    total = total + count
print(total)
```

```
counts = [4, 7, 4]
print(counts[3])
```

Record	Your evidence
Given input and fixed rule	

Record	Your evidence
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Lists and indexing. Explain why each step is justified.

Explain → apply → challenge

For counts = [4, 7, 4], give the **length**, valid nonnegative indices and value at **index** 1. Explain how the two **item**s with value 4 differ.

A learner loops with for count in counts and tries to read counts[count] for counts = [4, 7, 4]. Identify the first attempted **index** and error. Explain how to print each value directly.

Compare [1, 5, 2] and [5, 1, 2]. State their totals and first **item**s. Would storing only the total answer which first **item** was larger? Explain.

Explain which evidence supports your conclusion and which claim remains unproved.

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Python readiness portfolio. Keep an annotated decision and the evidence supporting it.

Add a labelled **list**s and **index**ing **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **list index** from the value stored at that position.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

Before running a script, sketch positions above the stored values. Test an empty **list**, a one-**item list** and a **list** with repeated values. These cases reveal different mistakes.

A total alone loses order: [4, 7, 4] and [7, 4, 4] both total 15 even though their first **item**s differ. Data checks should retain the detail needed for the question.

> **A learner uses counts[len(counts)] to read the final item. Explain the error for [8, 3] and give a valid nonnegative index. Explain why your repair still needs an empty-list check.**

Compare [1, 5, 2] and [5, 1, 2]. State their totals and first **item**s. Would storing only the total answer which first **item** was larger? Explain.

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete Lists and indexing case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

Positions and stored counts

TRAY POSITION	INDEX	VALUE
First	0	4
Second	1	7
Third	2	4

Equal stored values do not imply equal positions. There is no item at index 3.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

STUDENT LESSON · 14 OF 18

Try a new case independently

Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Distinguish a value from an index

Predict the three **output**s. If the second print uses values[3] instead, where does execution stop and which later **output** is not produced?

```
values = [4, 9, 2]
print(values[1])
print(values[-1])
print(len(values))
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Start with readings = [3, 0, 3, 2]. Replace the second **item** with 5 using its **index**. Write a loop that adds all updated readings and prints one final total. Trace every update of the total.

Then test the summing part alone on [] and [6].

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

Your lesson route

> Use this map to navigate Lists and indexing, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: **Index**es are positions, not values. A **list** of shelf counts is read by position.

By the end, I can	How I will show it
Distinguish a list index from the value stored at that position	Explain the deciding step, show a trace or test, and state one limit.
Read and update an item using a valid index	Explain the deciding step, show a trace or test, and state one limit.
Trace a loop through list values and test an empty list	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Lists and indexing without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Distinguish a **list index** from the value stored at that position

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Indexes are positions, not values. A **list** of shelf counts is read by position.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2.

1. Read two positions

2. Check **length** boundary

3. Draw positions

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Check **length** boundary **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2. **List** positions begin at zero.

A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Replace the second print with print(counts[3]). Which conclusion is supported by the supplied facts?

1. The first print shows 6, then **IndexError** stops the script before the **length** print.

2. We can decide the result without examining the changed **input** or the procedure.

3. This one example guarantees correct results for every future **input**.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: The first print shows 6, then **IndexError** stops the script before the **length** print. The attempted position does not exist.

Repair a missing building block

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: **List** the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Boolean conditions (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review From a paper algorithm to Python (python-bridge/1). Goal: Translate a short paper calculation into a Python script. Short practice: Match each paper quantity to its Python name.

Why does the spare count get added once, and what prints when spare is five?. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

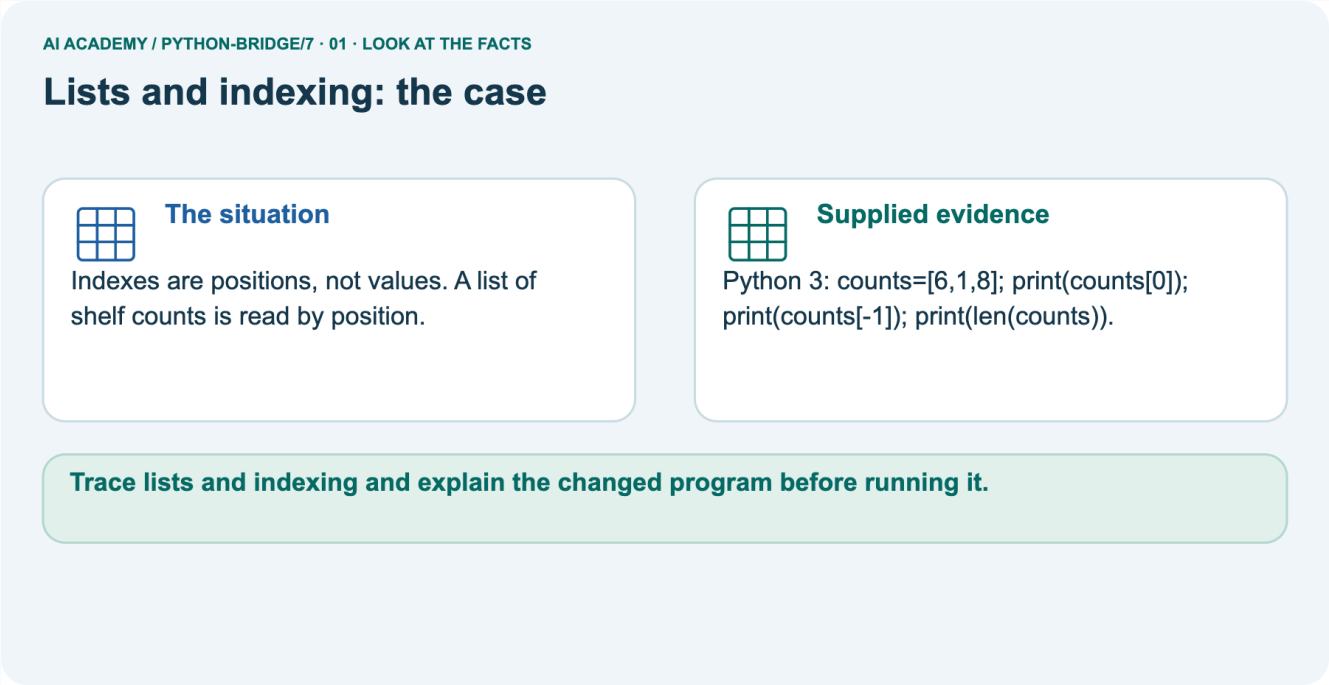
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each Lists and indexing graphic using its labels, deciding evidence and relevant limitation.

01. Lists and indexing: the case

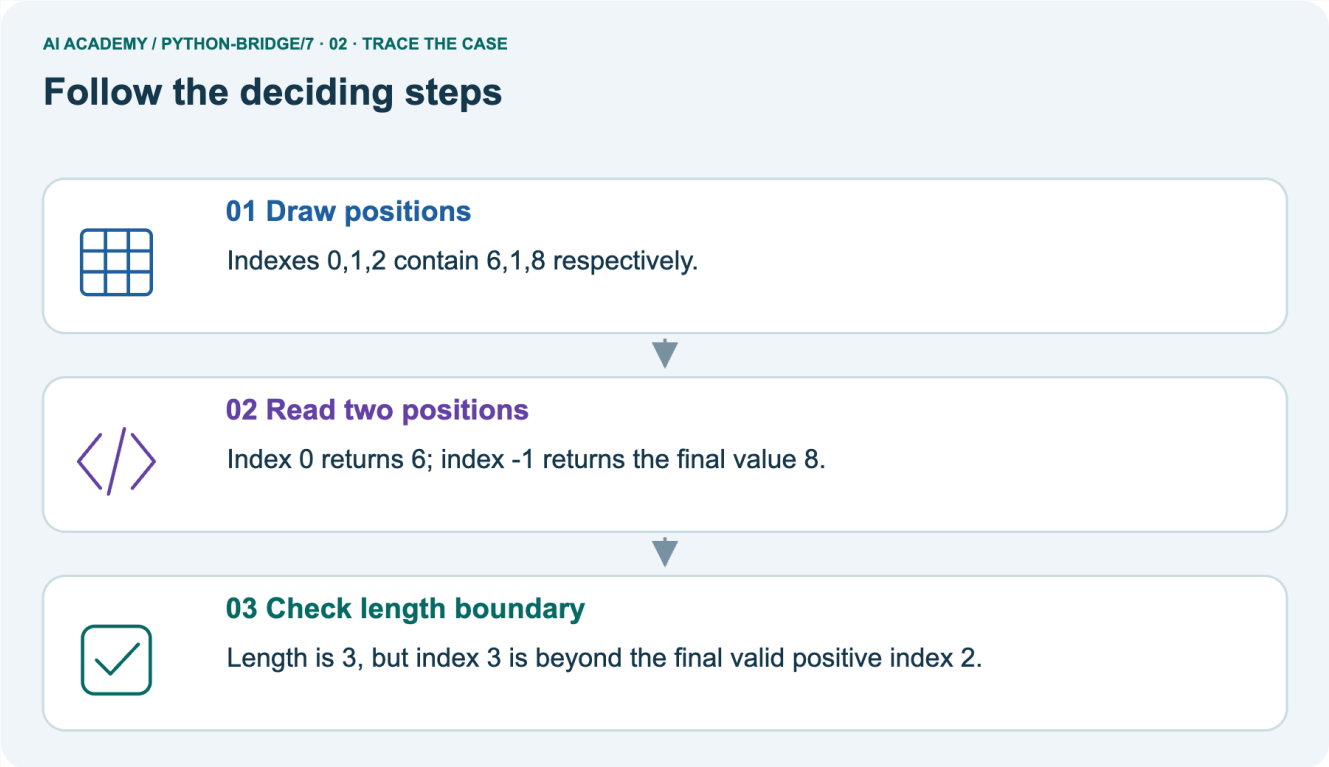


Follow the labelled picture with the matching explanation. Open the vector diagram

Indexes are positions, not values. A **list** of shelf counts is read by position. Supplied facts: Python 3: counts=[6,1,8]; print(counts[0]); print(counts[-1]); print(len(counts)).

> Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Draw positions: **Index**es 0,1,2 contain 6,1,8 respectively. → Read two positions: **Index** 0 returns 6; **index** -1 returns the final value 8. → Check **length** boundary: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2.

> Point to each numbered stage and explain which recorded **input** it uses.

03. Zero-based list positions

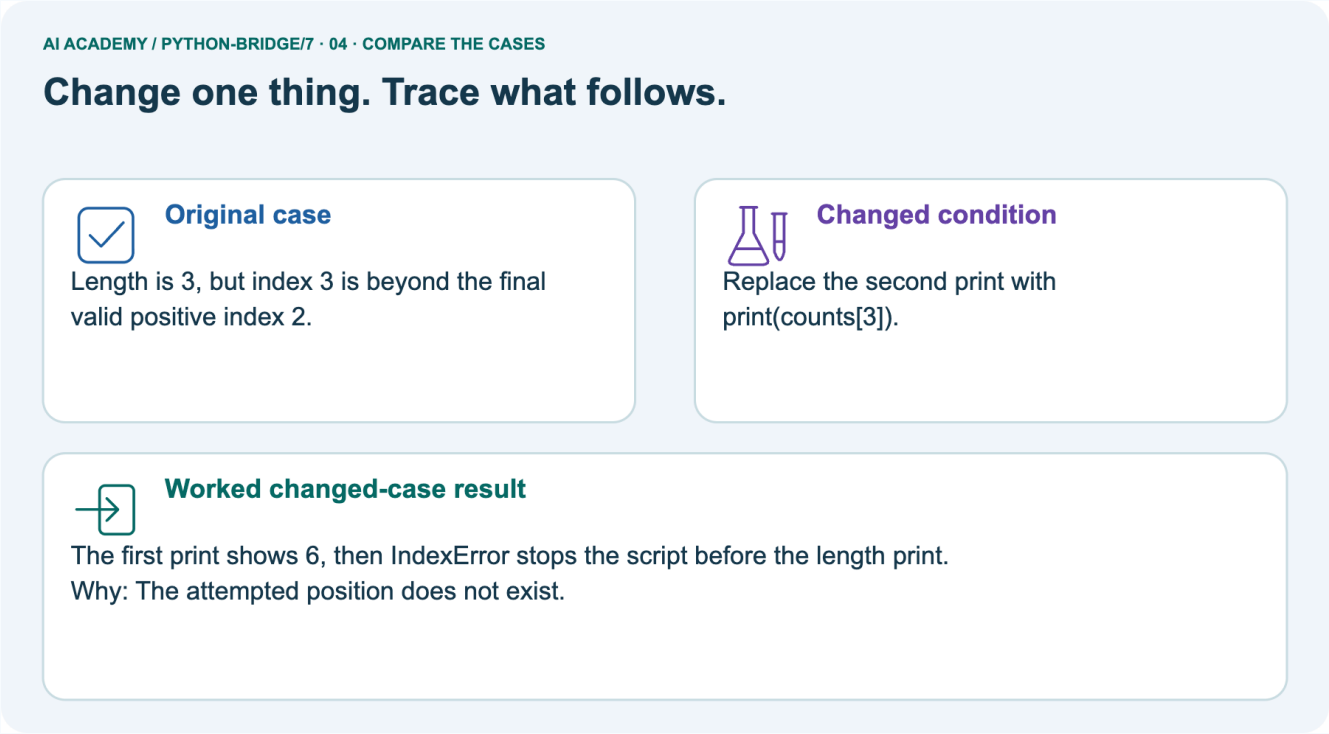


Follow the labelled picture with the matching explanation. Open the vector diagram

Zero-based **list** positions. Stored counts: **Index** 0 = 6, **Index** 1 = 1, **Index** 2 = 8.

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



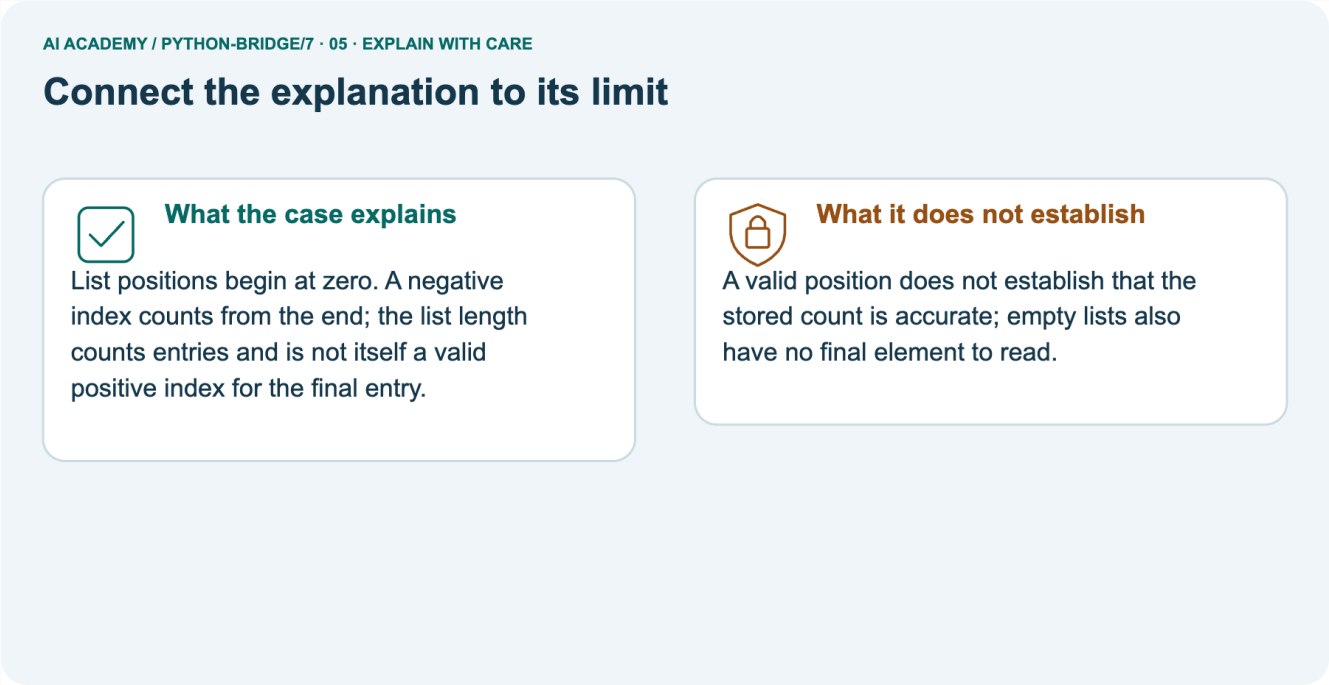
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: **Length** is 3, but **index** 3 is beyond the final valid positive **index** 2. Changed condition: Replace the second print with print(counts[3]).

Expected result: The first print shows 6, then **IndexError** stops the script before the **length** print. Reason: The attempted position does not exist.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



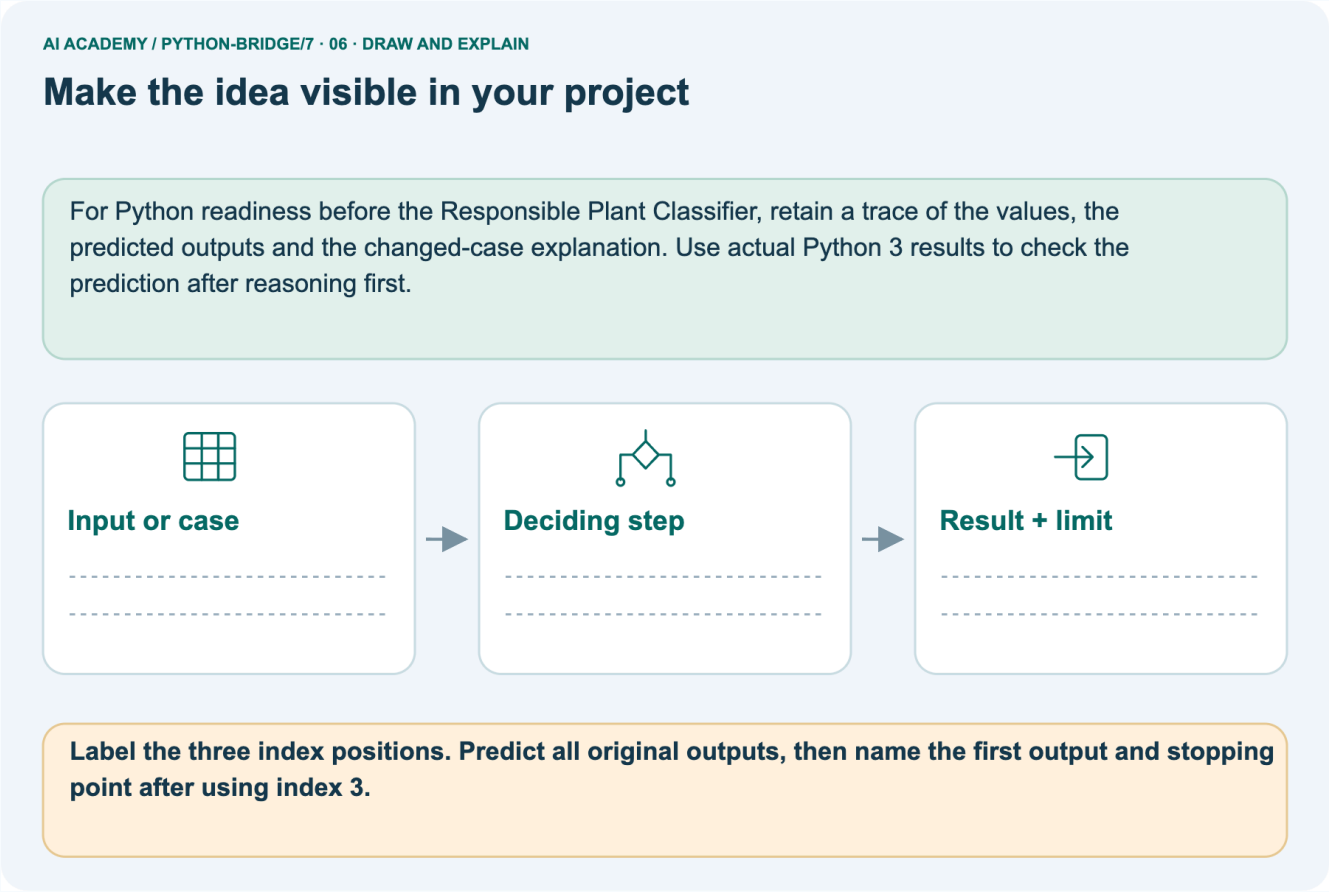
Follow the labelled picture with the matching explanation. Open the vector diagram

List positions begin at zero. A negative **index** counts from the end; the **list length** counts entries and is not itself a valid positive **index** for the final entry.

Limit: A valid position does not establish that the stored count is accurate; empty **list**s also have no final element to read.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Label the three **index** positions. Predict all original **output**s, then name the first **output** and stopping point after using **index** 3.

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.